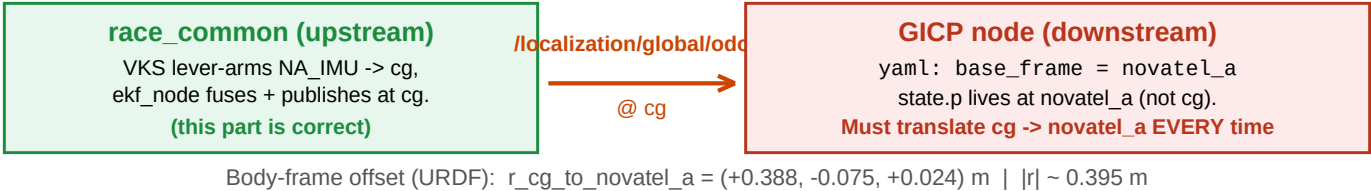


GICP Frame-Mismatch Bug

/localization/global/odom arrives at cg; GICP state lives at novatel_a; GICP handles the cg -> novatel_a translation INCONSISTENTLY (race_common's upstream compensation is correct -- this is a GICP-side bug, not a race_common bug.)



Three GICP code paths read this data. Their lever-arm handling differs:

Code path	Where (localization.cc)	Lever-arm?	Status
applyInitialPose First gt_odom message: state.p = s.p, state.q = s.q copied directly. Initial state seeded 0.39 m off.	callbackGtOdom ~L2367	NO	BUG
cross-check gt_pos_err = cand_p - gt.p . cand_p @ novatel_a, gt.p @ cg. ~0.39 m baseline bias for the whole run.	performLocalization ~L1864	NO	BUG
maybeSnapPoseToGT Caches T_base_gtbody = lookup_transform("novatel_a", "cg") on first GT msg; composes T_map_base.	snap helper ~L2495	YES	CORRECT

Why this is a GICP bug, not a race_common bug

race_common's compensation is correct under its own convention: VKS looks up the URDF TF NA_IMU -> cg and applies it to every per-source filtered_odom, then robot_localization's ekf_node fuses the result and publishes /localization/global/odom at cg. Anyone whose own state lives at cg consumes this topic with zero further work. The design is internally consistent and correct.

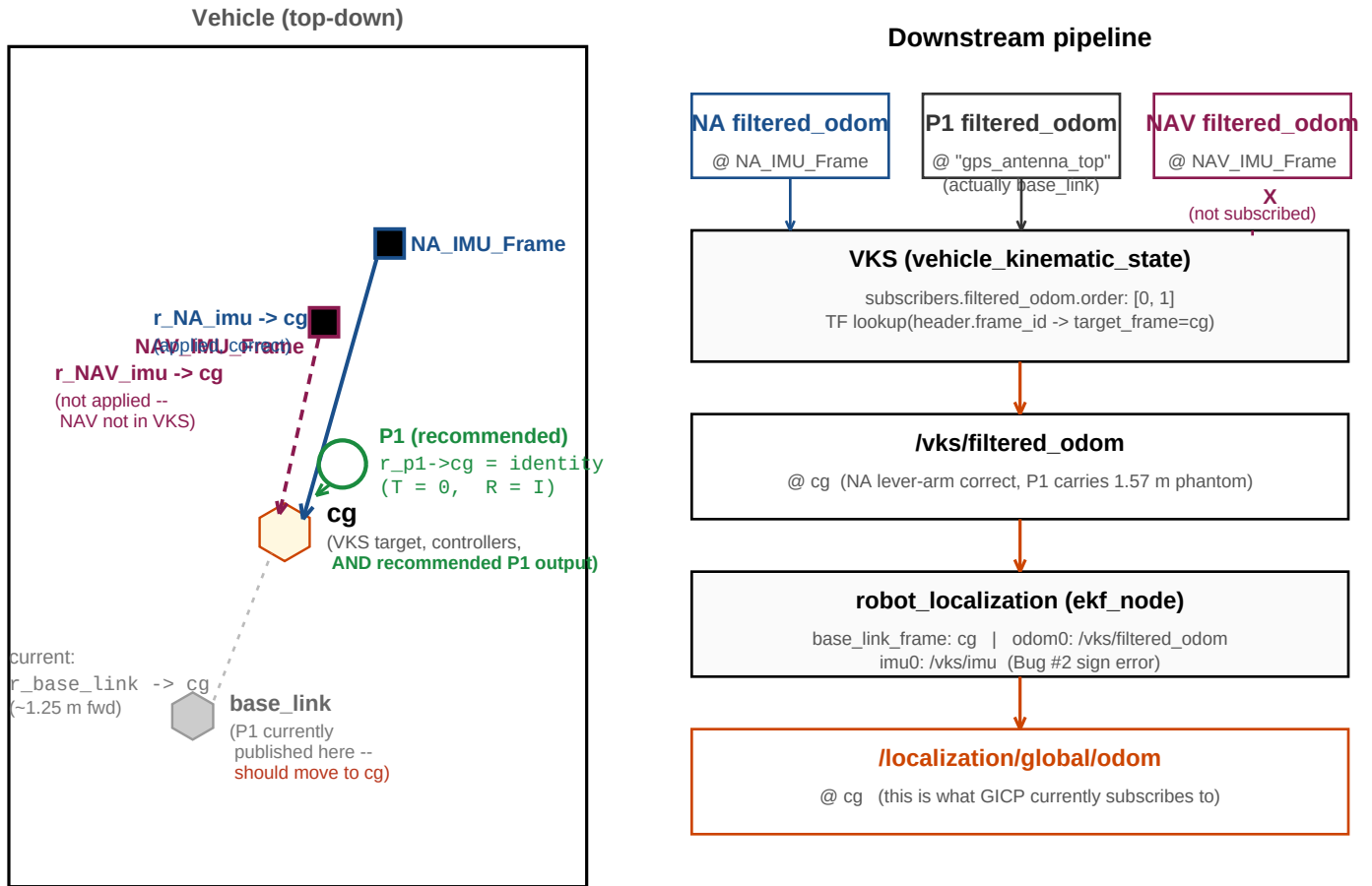
GICP's yaml puts state at novatel_a, NOT cg. So GICP -- and only GICP -- has to apply the cg -> novatel_a inverse translation in every code path that touches the gt_odom data. The snap helper (maybeSnapPoseToGT) does this correctly. The other two paths skip it, silently producing a 0.39 m offset.

Two fixes, in increasing risk + reward:

- (a) Local fix: apply the same cached T_base_gtbody_ in applyInitialPose and in the cross-check. Heals both bugs without changing the broader architecture.
- (b) Architectural fix (recommended): set localization/base_frame: cg in localization.yaml. All three paths' lever-arm operations degenerate to identity; no per-path code change needed. Requires fixing the dormant in-code IMU lever-arm sign in callbackImu first (it activates once base_frame != imu_frame; see Bug #2 / #4 in the report).

Downstream Pipeline: IMU --> base_link Lever-Arm in race_common

What VKS + robot_localization do to bring each source to a common body reference



Are the IMU --> body-reference transforms correctly applied?

Vendor firmware behavior (from official documentation):

NA (NovAtel PWRPAK7-E2): antenna -> IMU only. Publishes at NA_IMU_Frame.
NAV (VectorNav VN-310): antenna -> IMU only. Publishes at NAV_IMU_Frame.
P1 (Atlas FusionEngine): antenna -> body AND IMU -> body. Publishes at user-configured body frame.
(Atlas User Guide v1.6: "measure the lever arms from the vehicle body frame to BOTH Atlas and the GNSS antenna". The "vehicle body frame" is whatever the team sets in device.lever_arm.)

Recommended config (the cg-aligned design):

NA: apply VKS lever-arm novatel_a -> cg (~0.39 m back). Done today; correct.
NAV: enroll in VKS, apply vectornav -> cg (~0.51 m back). TF exists; not subscribed.
P1: reconfigure device.lever_arm so P1 outputs DIRECTLY at cg (not base_link).
Then VKS lever-arm for P1 is identity ($T = 0, R = I$) -- no TF translation needed.
This makes P1 fully aligned with race_common's controllers, MPC, and EKF, all of which already operate at cg (see race_vehicle_controller, race_mpc_core, vehicle_model).

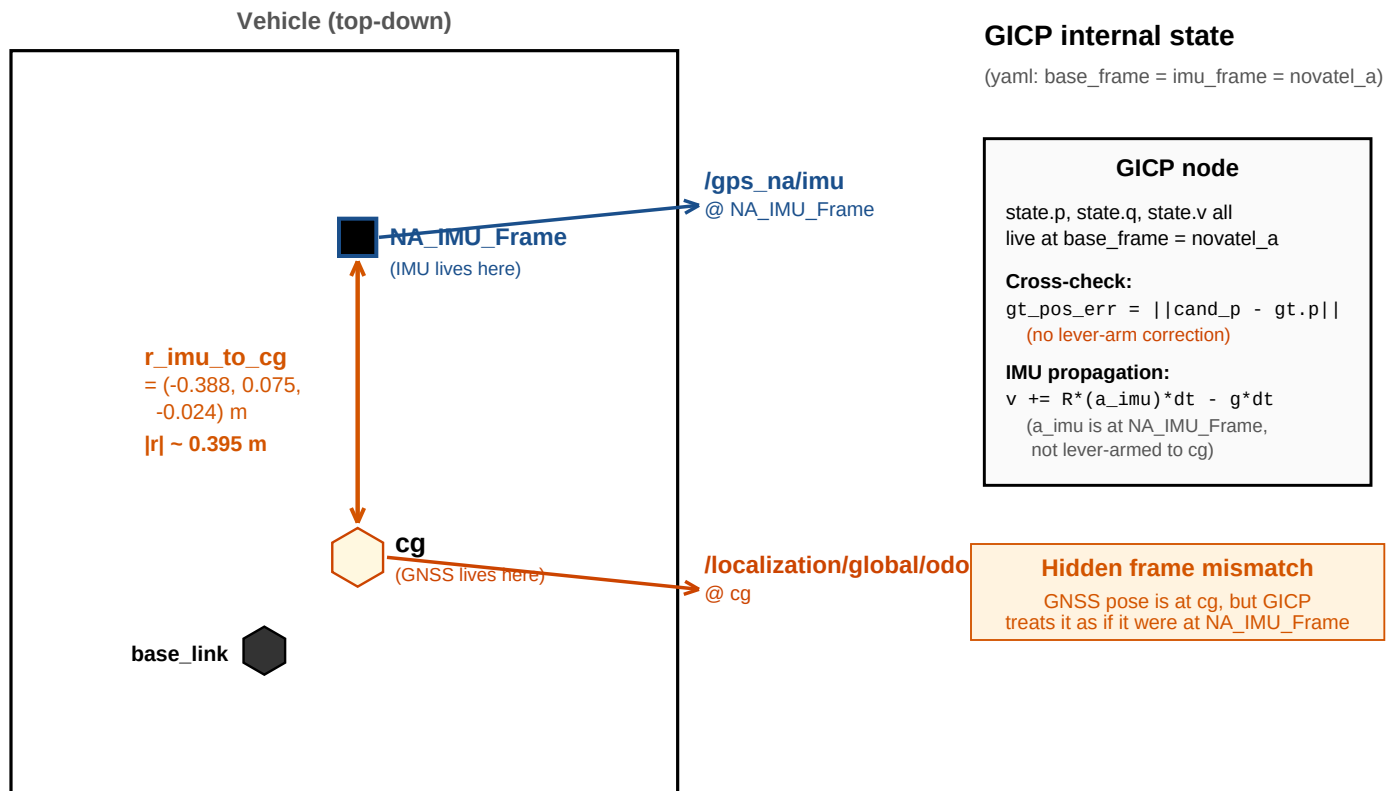
Current state of P1 (what to migrate FROM):

The team set device.lever_arm so P1 publishes at base_link, NOT cg (verified on vehicle).
Worse, the FusionEngine ROS driver labels every message with frame_id="gps_antenna_top".
VKS trusts the (wrong) label and applies a gps_antenna_top -> cg TF, not base_link -> cg.
The phantom shift is 1.57 m total (1.27 m horizontal + 0.92 m vertical) -- Bug #3.

Fix order: (1) point P1 at cg in device.lever_arm, (2) update frame_id="cg" in atlas.param.yaml.
After both, P1 is symmetric with NA/NAV at the body-reference layer, and Bug #3 disappears.

GICP Frame Mismatch: GNSS @ cg vs IMU @ NA_IMU_Frame

GICP today consumes /localization/global/odom and /gps_na/imu, which live at DIFFERENT physical points on the car



Consequences of the GNSS @ cg / IMU @ NA_IMU_Frame mismatch

Both upstream sources arrive at GICP, but they sit at different points on the rigid body. GICP's yaml sets `base_frame == imu_frame == novatel_a`, so the IMU side is internally consistent (state.v is computed from `a_imu` at `NA_IMU_Frame`, and the state tracks `NA_IMU_Frame` too). The GNSS side is not. Four downstream effects:

(1) Cross-check baseline error (high visibility, no correctness impact on its own).

$gt_pos_err_m = ||cand_p - gt.p||$ with no TF correction. `cand_p` tracks `NA_IMU_Frame`; `gt.p` is at `cg`. Reported error has a constant $\sim 0.39 \text{ m}$ forward bias (plus smaller Y/Z terms) even when GICP is perfectly localized. Operators reading SCAN DEBUG can't distinguish biased diagnostic from real divergence.

(2) Recovery snap is OK (lever-arm-corrected inside the snap helper).

`maybeSnapPoseToGT` composes `T_map_base = T_map_gtbody * inv(T_base_gtbody)` by looking up the TF `base_frame -> msg.child_frame_id` ("cg" for /loc/global/odom). When the snap fires, it correctly translates the cg pose into a `novatel_a` pose.

(3) IMU velocity is at NA_IMU_Frame; GNSS twist is at cg. Consistent with state but not with each other.

`state.v` lives at `novatel_a`. `/loc/global/odom.twist` lives at `cg`. During yaw at rate wz , the two differ by $wz \times r_{imu_to_cg}$. At 1 rad/s : $|dv| \sim 0.39 \text{ m/s}$ perpendicular to body x. Any module that compares the two will see this as a phantom slip angle.

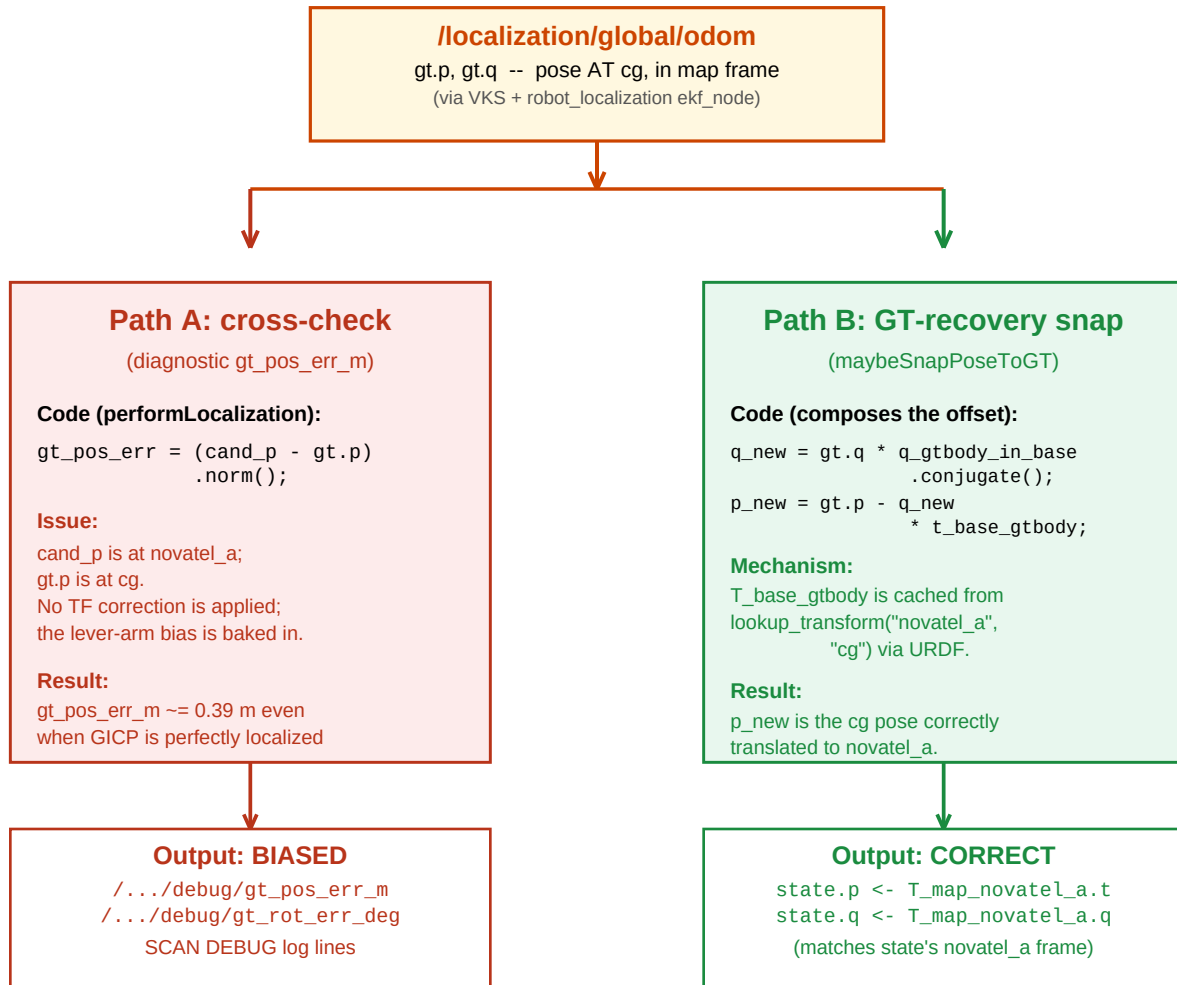
(4) Bug #1 fix (base_frame -> cg) activates the dormant in-code lever-arm.

Once `base_frame != imu_frame`, the `t` in `ang_vel_bl.cross(ang_vel_bl.cross(t))` becomes non-zero. The current code uses `+= cross()`, which inverts the centripetal sign (same defect as Bug #2 in VKS). Left unfixed it adds $\sim wz^2 \times 0.39 \text{ m/s}^2$ of wrong-direction body-x acceleration during cornering ($\sim 3.5 \text{ m/s}^2$ at race-corner yaw).

Required sequence: fix the in-code lever-arm sign in `callbackImu` BEFORE switching `base_frame` to `cg`, otherwise the Bug #1 cure introduces a new acceleration bug.

Same GT input, two code paths: snap is correct, cross-check is biased

Both consume /localization/global/odom @ cg, but only one applies the URDF lever-arm to bring it to novatel_a



Why the two paths diverged (and what it tells us)

The snap path was written with care: someone noticed that the GT odom publishes with child_frame_id = "cg" while GICP tracks novatel_a, so the snap helper caches the URDF TF base_frame -> child_frame_id on the first GT message and composes T_map_base via inv(T_base_gtbody). When the snap fires, it correctly places the vehicle's novatel_a origin in map. No silent error.

The cross-check path, written separately, subtracts the two positions directly without the same TF correction. The result -- gt_pos_err_m -- carries a constant ~0.39 m bias for the whole run. Operators reading SCAN DEBUG can't tell biased diagnostic from real divergence without back-computing the offset.

Lesson: when state.p lives at one frame but GT lives at another, every comparison in the code must apply the lever-arm. The snap remembered. The cross-check did not. Any future code that compares GICP state to GT (velocity divergence, smoothing fuse, EKF update) has to remember too -- which is fragile.

Two fixes:

- Apply the same TF correction in the cross-check: subtract (gt.p + R_map_gt * t_gt_to_base) instead of gt.p directly, using the cached T_base_gtbody the snap already maintains.
- Change localization/base_frame to cg. State, cross-check, snap, and downstream consumers all align; the snap's TF lookup degenerates to identity. This is the architectural fix recommended in Bug #1 of the bug report. Requires fixing the dormant in-code IMU lever-arm sign error in callbackimu first (Bug #2 / #4).